

Manual de administración corta de MultiBioTraductor

1. Requisitos del sistema

- **Servidor / máquina local:**
 - Python 3.x instalado.
 - Sistema operativo compatible (Linux, Windows, macOS).
 - Opcional: Docker y docker-compose instalados.
- **Dependencias:**
 - click==8.1.3
 - colorama==0.4.5
 - Flask==2.1.2
 - gunicorn==20.1.0
 - itsdangerous==2.1.2
 - Jinja2==3.1.2
 - MarkupSafe==2.1.1
 - python-decouple==3.6
 - Werkzeug==2.2.2

2. Instalación en entorno local (sin Docker)

1. Clonar o copiar el repositorio del proyecto.
2. (Opcional pero recomendado) crear un entorno virtual:
`python -m venv venv`

`source venv/bin/activate` # Linux/Mac

`venv\Scripts\activate.bat` # Windows
3. Instalar dependencias:
`pip install -r requirements.txt`
4. Ejecutar la aplicación:
`python app.py`
5. Abrir un navegador en:
 - <http://localhost/> o
 - <http://localhost:80/> según configuración. o
 - <https://ciencias.medellin.unal.edu.co/gruposdeinvestigacion/prospeccionydisenobiomoleculas/herramientas/traductor.html> en la página del grupo.

3. Despliegue con Docker

1. Construir la imagen:
`docker build -t multibiotraductor .`
2. Levantar los servicios con docker-compose:
`docker-compose up -d`

o, según el archivo de producción:
`docker-compose -f docker-compose-prod.yml up -d`
3. Confirmar el puerto mapeado en `docker-compose.yml` (ej. 80 o 8000) y acceder:
 - `http://<ip-del-servidor>:<puerto>/`

4. Ficheros de configuración relevantes

- **requirements.txt**
Lista de dependencias Python.
Usar `pip freeze > requirements.txt` para actualizarlas si se añaden nuevas.
- **Dockerfile**
Define:
 - Imagen base (Python).
 - Copia del código al contenedor.
 - Instalación de dependencias.
 - Comando de arranque (Flask o gunicorn).
- **docker-compose.yml / docker-compose-prod.yml**
 - Orquestan servicios.
 - Definen variables de entorno, puertos expuestos, volúmenes, etc.
- **Procfile**
 - Usado por plataformas como Heroku:
 - Ejemplo: `web: gunicorn app:app.`

5. Mantenimiento y operación

- **Revisión de logs:**
 - Ejecución directa: mensajes de Flask en la consola.
 - Docker:
`docker logs <nombre_contenedor>`
- **Actualización de dependencias:**
 - Editar `requirements.txt` cuando sea necesario.
 - Volver a construir la imagen Docker si se usan contenedores.
- **Respaldo:**
 - La aplicación no guarda datos en base de datos; es esencialmente estática.
 - El “backup” consiste en mantener el código versionado (Git).
- **Comprobación tras cambios:**
 - Después de hacer cambios en `script.js`, plantillas o estilo, reiniciar el servidor o el contenedor.
 - Probar manualmente:
 - Traducción de una secuencia corta.

- Carga de un archivo FASTA.
- Descarga de resultados.

6. Consideraciones de seguridad

- Si se expone en internet:
 - Usar un proxy inverso (Nginx/Apache) con HTTPS.
 - Ejecutar Flask detrás de un servidor de producción (gunicorn, uWSGI).
- Mantener las versiones de Python y dependencias actualizadas.
- Como la lógica biológica se ejecuta en el cliente, el servidor no almacena secuencias; aun así, es recomendable:
 - Registrar accesos y errores.
 - Proteger la infraestructura (firewall, actualizaciones del sistema).

7. Autores y créditos

- **Autora principal:** *Hinara Pastora Sánchez Mata*. Información de contacto: 3044822338, hisanchezm@unal.edu.co
- **Crédito institucional:** “Desarrollado por la línea de interacciones moleculares — **Biología Funcional**”.